

Triadic Runtime Specification

v1.0-draft

Status: **Sections 1–9 frozen as of this draft.** Section 10 is intentionally a stub — this specification separates the normative runtime from any reference implementation, and no reference runtime has been built against it yet.

Amendment process. Implementation work **MUST NOT** modify Sections 1–9 directly. A contradiction or gap discovered during implementation is filed as a counterexample against the frozen text — proposed amendment, proof or attack, then a new version (v1.1, etc.) — never a silent edit. This preserves the distinction between what the specification currently claims and what implementation experience suggests it should claim next.

This document was produced through adversarial construction, not first-principles design — see the companion Design Record for the full derivation history, including every counterexample that shaped a rule in Sections 4–9.

1. Scope

1.1 What this is

This document specifies a minimal runtime for recording and coordinating around three kinds of durable facts — observations, commitments, and intentions — across nodes that may be offline, mutually distrusting, and under no shared authority.

The runtime's job is narrow: preserve a faithful, causally-linked, tamper-evident history of what was observed, committed, and intended, and make conflicts in that history visible rather than silently resolved. Everything else — what a conflict *means*, who *should* win, what a workflow, contract, identity, or reputation *is* — is computed by applications querying that history. The runtime does not compute it and does not attempt to.

1.2 What this is not

This is **not**:

- A consistency system. It does not guarantee a single global order of events (see 1.3, AP boundary).
- An enforcement engine. It does not prevent unauthorized, incorrect, or harmful actions from happening in the world — it only guarantees that authorization claims made *inside* the runtime are explicit and traceable.
- A semantic system. It has no built-in understanding of any domain (finance, agriculture, medicine, etc.) — see Section 6, Local Meaning.
- A replacement for consensus protocols (Raft, Paxos) in domains that require strict linearizability. See 1.3.

1.3 The AP Boundary

This runtime is **available-and-partition-tolerant (AP) by construction**, not consistent-and-partition-tolerant (CP). Every node must be able to make progress — record observations, commitments, and intentions — using only locally available information (Axiom 3, Local Sufficiency). This is incompatible with protocols like Raft that require contacting a majority of nodes before a write is considered valid.

This is a deliberate, permanent design boundary, not a gap to be closed later. Systems that require strict linearizable consistency (e.g., a single authoritative leader that must never permit even momentary conflicting writes) are out of scope for this runtime and should be built as adapters that consume from it, not as extensions of it.

1.4 The Central Claim

The runtime records history. It never records truth.

Observations are beliefs, not facts. Commitments are institutional declarations, not descriptions of independent reality. Intentions are goals, not guarantees. None of the three primitives claims to be true in any sense the runtime itself can verify — truth, correctness, and meaning are always computed above the runtime, by applications interpreting the graph. This single sentence explains most of the boundary theorems in Section 9 and should be treated as the lens for resolving any ambiguity not otherwise covered by this document.

1.5 Interpretation

The runtime never interprets the contents of a payload. It guarantees integrity, provenance, causality, authorization, and conflict visibility — nothing about what a payload *means*. Whether a schema is sensible, whether a contract's terms are fair, whether a delegation makes business sense: none of this is evaluated by the runtime. A conforming implementation that rejects a syntactically and causally valid record because its *content* seems wrong is not conforming — it has silently taken on an interpretive role that belongs to applications (see Section 6, Local Meaning).

1.6 Conformance

An implementation conforms to this specification if and only if it satisfies every Axiom (Section 4) and every Well-Formedness Rule (Section 6), and passes every Conformance Test (Section 7). Derived Theorems (Section 8) and Boundary Theorems (Section 9) are not independently checkable — they are consequences of conformance, included for explanatory purposes.

Rejecting a record for reasons outside these three categories (Axioms, Well-Formedness Rules, Conformance Tests) — for instance, because a payload's content seems implausible or unwise — is itself a conformance violation (see 1.5, Interpretation).

2. Design Principles

Non-normative. These principles are not independently testable and impose no requirements of their own. They explain why the Axioms and Well-Formedness Rules in Sections 4–6 take the shape they do, and should be used to resolve ambiguity when extending this specification.

2.1 Faithful History over Correctness. The runtime's job is to record what happened, who claimed what, and when — not to guarantee that what happened was correct, authorized in the world, or wise. Correctness is computed above the runtime, by policy.

2.2 Explicit over Implicit. Authority, semantics, delegation, and conflict must always be represented as explicit records or explicit absence of records. The runtime never infers permission, meaning, or resolution silently.

2.3 Local First. A node's ability to make progress must never depend on contacting another node. Synchronization improves what a node knows; it is never a precondition for a node acting on what it already knows.

2.4 Additive Evolution. State only ever grows, by supersession and causal extension. Nothing is ever deleted or mutated in place. History is append-only, even when it records a mistake, a reversal, or a conflict.

2.5 Policy Neutrality. The runtime stores history and makes it queryable. It never decides which of two conflicting claims is right, which goal takes priority, or which interpretation of a contract is binding. Every such decision belongs to an application-level policy reading the graph.

2.6 Minimal Elevation. A concept SHALL only be added to the ontology (Section 3), given axiom status (Section 4), or given a dedicated well-formedness rule (Section 6) when it cannot be expressed as a composition of existing primitives and payloads. This is not merely a stylistic preference — it is the methodology that produced this specification. Identity, Reputation, Contracts, Workflow, Multi-Agent Coordination, Capability Security, and Semantics were each tested against this principle by deliberate adversarial construction (Section 7) and each reduced to compositions of the three primitives rather than requiring extension. Any future extension to this specification should be held to the same standard: attempt the reduction first, and only elevate when the reduction demonstrably fails.

3. Ontology

3.1 Overview

The runtime recognizes exactly three persistent record roles. These are roles — verbs, not domain objects — distinguished by **direction of fit**: the relationship between the record and the reality it concerns.

Primitive	Direction of fit	On mismatch with reality
Observation	word $\rightarrow$ world (belief must fit reality)	the belief updates
Commitment	world $\rightarrow$ word (reality is constituted by the declaration)	nothing updates; only a new commitment can supersede
Intention	world $\leftarrow$ effort (a goal aimed at a future state)	the action updates; the goal persists or is explicitly abandoned

No domain object (a "person," an "order," a "contract," a "workflow") is ever a fourth primitive. Every domain concept is either (a) a payload carried by one of these three record types, or (b) a transient interpretation — a query result — computed over the graph of records. See Section 7 for a catalogue of domain concepts (identity, reputation, contracts, workflows, multi-agent coordination, capability security) constructed entirely from these three roles, and Section 8 for the theorems this produces.

3.2 Observation

Purpose. To record a belief about the state of the world, held with some confidence, subject to revision as new evidence arrives.

Direction of fit. Word $\rightarrow$ world. The observation must fit reality; when it doesn't, the observation is wrong, not the world.

Identity. Requires **provenance**, not authority. A sensor or a witness does not need permission to report what it saw — it only needs to be identifiable so the observation can be weighted and traced. This is the weakest identity requirement of the three primitives, by design (see 1.5, Interpretation — the runtime does not gatekeep who is allowed to hold a belief).

Lifecycle. Write-once. Never mutated. A revised belief is a new observation that supersedes the prior one; the prior observation remains in history, unmodified.

Serialization requirements:

- **Subject** — what the observation is about
- **Assertion** — the claimed state
- **Author/provenance** — who or what produced this observation
- **Origin** — witnessed (a person, sensor, or API directly reported this) or inferred (the runtime or an application derived this from a rule, such as an expiry deadline; see 6.3)
- **Timestamp**
- **Evidence** (optional) — supporting material or a reference to what grounds the claim
- **Confidence** (optional, may be a fixed value or a function such as a decay curve)

3.3 Commitment

Purpose. To constitute an institutional fact by the act of being made — a commitment does not describe an independently existing reality, it creates one (a balance, an identity claim, a delegation, a contract term).

Direction of fit. World $\leftarrow$ word. The declaration itself is what makes the fact true; reality does not get a vote. A commitment is not merely a declaration — it is a declaration **whose validity is mechanically checkable**: any party, at any time, can determine whether a given commitment is valid solely by traversing the causal graph back to a genesis record or an authorizing prior commitment, and checking it against the applicable well-formedness rules (Section 6). A commitment whose validity cannot be mechanically determined this way is malformed, regardless of how plausible its content is.

Identity. Requires **authorization**, not just provenance. Every commitment must carry a reference to what authorizes it: either (a) a causal link to a prior commitment that delegates the right to make it, or (b) cryptographic self-authorization for genesis records (see 4.2). An unauthorized commitment is malformed and is not a valid part of the graph (see 6.1).

Lifecycle. Write-once. Never mutated or deleted. Can only be superseded or countered by a later, causally-linked commitment (a revocation, an amendment, a correction).

Serialization requirements:

- **Subject**
- **Assertion** — the fact being constituted
- **Authorization** — see Identity, above
- **Timestamp**
- **Payload type** (see Section 5) — what kind of institutional fact this is: identity, delegation, schema, policy, contract term, etc.

3.4 Intention

Purpose. To record a desired future state, together with references sufficient to prove that its author was authorized to create it — the authorization is never contained within the intention, only referenced (see 6.1, Authorization Traceability; this distinction was one of the central findings of the derivation process behind this specification).

Direction of fit. World $\leftarrow$ effort. An intention aims to reshape a future state that doesn't yet exist. On mismatch — the world isn't becoming what was intended — the action updates, not the intention itself, and not the world by fiat.

Identity. Requires **authorization**, on the same footing as Commitments, not a weaker standard. This is a corrected symmetry, not an original design choice: earlier drafts of this ontology allowed intentions to be justified by mere causal reference to an observation or another intention, which permits authorization forgery (see 7.x, Capability Security). An intention record is well-formed only if, at creation time, a valid authorization path can be derived through the commitment graph. The runtime makes no claim that actions taken

outside the runtime are authorized or prevented — it guarantees only that authorization claims represented inside the runtime are explicit, traceable, and fail safe (see 6.1, 8.1.4).

Lifecycle. The only primitive with internal state transitions: $\boxed{\text{active} \rightarrow \{\text{completed} \mid \text{cancelled} \mid \text{expired}\}}$. This is a real structural asymmetry among the three primitives. Once an intention reaches a terminal state, it cannot reopen — a retry or continuation is always a new intention, causally referencing the terminal one. Every intention that reaches a terminal state **must** emit at least one observation recording the outcome (Axiom 4, Closure) — this is what allows the runtime to be learned from, rather than being a write-only log of unexamined goals.

Serialization requirements:

- **Subject**
 - **Goal / success criteria** — explicit conditions under which the intention is considered achieved
 - **Authorization** — a traceable path through the commitment graph (see Identity, above)
 - **Expiry or cancellation condition** — explicit
 - **Timestamp**
 - **Causal antecedents** — the observations, commitments, or (within the same authority) parent intentions that motivated this intention
-

4. Core Invariants

Working title for this section is "Core Invariants," not "Axioms," until Section 10 is complete. An entry earns the label "Axiom" only if, after the full specification and reference runtime exist, it cannot be derived from the others. This is a stricter standard than asserting axiom status at draft time.

Each invariant below contains exactly one normative requirement. Where an exception or special case exists, it is stated as a separate defined term or deferred to Section 6 (Well-Formedness Rules), never folded into the invariant itself as an "unless" clause — mixing a requirement with its exceptions is the failure mode this section is written to avoid (see the correction to Causality, 4.2, for a case where this discipline was applied retroactively).

4.0 Definitions used in this section

Genesis Record. A record with no causal predecessors. Validity for a genesis record is established by cryptographic self-authorization rather than by reference to a prior record (see 6.1).

Causal predecessor. A record R2 has causal predecessor R1 if R2's creation directly depended on R1 existing — R1 was read, cited as evidence, or referenced as authorization when R2 was made.

Terminal state. For an Intention: $\boxed{\text{completed}}$, $\boxed{\text{cancelled}}$, or $\boxed{\text{expired}}$. A record in a terminal state cannot transition to any other state.

4.1 Immutability

Normative statement. A conforming runtime MUST NOT permit any existing record to be modified or deleted after creation.

Rationale. History that can be silently rewritten cannot be trusted by a party who wasn't present when the rewrite happened — which is the normal condition for every party in an offline-first, multi-authority system. Making immutability absolute, rather than "immutable except for corrections," is what makes Explainability (8.1.1) and Convergence (8.1.2) derivable rather than merely likely.

Conformance test. Given any valid record R accepted by the runtime at time T, no operation available to any party — including the record's own author — may change R's content or remove R from the store at any time $T' > T$. A "correction" MUST be observable in the store as a distinct, new record causally referencing R, with R unchanged and still present.

Notes. Interpretations are not records. Immutability applies only to persistent records stored by the runtime. Values computed from those records at query time — for example confidence, reputation, contract validity, or policy evaluation — are transient interpretations, produced fresh on each query, and are therefore outside the scope of this invariant. The stored graph never changes; only a projection over it does.

4.2 Causality

Normative statement. A conforming runtime MUST require that every record either be a Genesis Record or reference one or more causal predecessors.

Rationale. This invariant is deliberately structural only. Earlier drafts of this specification stated causality as "every record references what justifies its existence," which conflated *structural* dependency (what this record was built from) with *authorization* (whether the record's author was permitted to build it). That conflation is exactly what the Capability Security derivation (7.x) exploited: a graph can satisfy causal reference while containing a record its author had no right to make. Causality answers only "is this graph structurally coherent"; Authorization Traceability (6.2) answers "was this record permitted." Keeping them as two separate, independently checkable requirements is what makes both checkable.

Conformance test. For any non-genesis record R in the store, the runtime MUST be able to produce at least one record R' such that R references R' as a causal predecessor. A record with zero causal predecessors that is not marked and validated as a Genesis Record (4.0, 6.1) MUST be rejected.

Notes. Causality does not require that the referenced predecessor be *authorized* — only that it *exist* and be *referenced*. See 6.2 for the authorization requirement layered on top of this for Commitments and Intentions specifically.

4.3 Local Sufficiency

Normative statement. A conforming runtime MUST permit any node to create a valid Observation, Commitment, or Intention using only information already present on that node, without contacting any other node.

Rationale. This is the invariant that makes the runtime AP rather than CP (see 1.3). A node that must contact a peer or a majority before it can record anything cannot function offline, which is a hard requirement for the intended deployment environment (intermittent connectivity, no assumed backbone). This invariant is also what rules out consensus protocols like Raft as valid conforming implementations of this runtime — not as an oversight, but as the direct, intended consequence of this requirement.

Conformance test. Simulate a node with zero network connectivity for an arbitrary duration. The node MUST remain able to append well-formed Observations, Commitments (where it holds sufficient local authorization), and Intentions (where it holds sufficient local authorization) throughout that period. Synchronization, when it later occurs, MUST only ever add records the node did not previously have — it MUST NOT be a precondition for any operation performed during the offline period.

Notes. Local Sufficiency governs the *creation* of records, not the *completeness of belief*. A node offline for three weeks may hold a stale or incomplete view of the graph — that is expected and permitted. What is not permitted is the node being unable to act at all until it reconnects.

4.4 Closure

Normative statement. A conforming runtime MUST require that every Intention which reaches a terminal state (4.0) has at least one Observation causally referencing it, recording the outcome.

Rationale. Without this invariant, the runtime can act on the world (via intentions) but never learn what happened — a purely reactive system with no feedback loop, unable to improve or even detect its own failures. Closure is what allows forecasting, fraud detection, or any other model built above the runtime to have a labeled dataset of intended-versus-actual outcomes to learn from.

Conformance test. For any Intention record I that transitions to `completed`, `cancelled`, or `expired`, the runtime MUST reject the transition unless at least one Observation exists (or is submitted atomically with the transition) that causally references I. This applies uniformly regardless of whether the terminal state was reached by direct action, explicit cancellation, or inferred expiry (6.3).

Notes. Closure guarantees only that *some* observation exists recording *an* outcome — it makes no claim about whether the outcome was favorable, or whether a parent intention's completion depends on its children's. Aggregation across an intention hierarchy is a query, not a runtime guarantee (see 9.x, Closure Is Strictly Local).

4.5 Non-Silent Conflict

Normative statement. A conforming runtime MUST preserve both records, unresolved, when two records are concurrent, causally derived from the same predecessor, and mutually exclusive in their assertion about a shared subject.

Rationale. A runtime that silently picks a winner between two conflicting records (via last-write-wins, timestamp priority, or any other implicit rule) hides the fact that a conflict occurred at all — which is a strictly worse outcome for every downstream party than surfacing the conflict and letting a policy layer resolve it deliberately. This invariant does not specify how conflicts should be resolved; it only forbids resolving them invisibly.

Conformance test. Given two records R1 and R2, both descending from the same causal predecessor P, both asserting mutually exclusive states about the same subject, and created without either being aware of the other (no causal path from R1 to R2 or R2 to R1): the runtime MUST retain both R1 and R2 in the store, and a query over the subject in question MUST surface both, rather than returning only one.

Notes. This invariant applies uniformly to conflicting Commitments (e.g., double-spend), conflicting Observations (e.g., two contradictory sensor readings), and conflicting Intentions (e.g., two agents claiming the last unit of a resource) — see 7.x for worked cases of each.

5. Record Model

5.1 Record Envelope

Every record accepted by a conforming runtime — regardless of primitive type — consists of an **envelope** and a **payload**. The runtime validates the envelope. It does not, and must not, interpret the payload (5.3).

The envelope MUST contain:

- **Identity** — a unique identifier for this record
- **Primitive type** — `observation`, `commitment`, or `intention`
- **Author** — the identity that created this record
- **Timestamp**
- **Causal references** — zero (genesis only) or more references to causal predecessors (4.2)
- **Authorization reference** — required for `commitment` and `intention` records; either a reference to an authorizing prior commitment or a self-authorization proof for genesis records (6.1)
- **Signature** — cryptographic proof that the declared author produced this record
- **Payload reference** — a pointer to the payload content and its type (5.2)

Primitive-specific envelope fields (Origin for Observations; Lifecycle state for Intentions) are as specified in Section 3.

5.2 Payload Types

The payload is where domain content lives. This specification defines the following payload types as commonly needed across constructions in Section 7; it does not limit implementations to only these types.

- **Schema** — defines a vocabulary (terms, units, relationships) that other payloads may reference for interpretability (6.4)
- **Delegation** — grants a capability from one identity to another, subject to 6.2
- **Capability** — a scoped, time-bounded permission grant, of which Delegation is one instance
- **Policy** — an application-level rule for interpreting conflicts, priorities, or aggregations; stored for auditability but never executed by the runtime itself
- **Domain payload** — any application-specific content (a balance, an inventory count, a contract term) that carries no meaning to the runtime beyond its structural role

New payload types may be introduced by any application without modifying this specification or requiring runtime changes, provided they conform to the envelope requirements in 5.1.

5.3 Payload Independence

The runtime SHALL validate record structure but SHALL NOT interpret payload semantics.

This is the normative statement underlying Section 1.5 (Interpretation). A conforming runtime verifies signatures, causal references, authorization chains, immutability, and conflict visibility — all properties of the envelope and the graph. It has no facility for, and must not implement, evaluation of what a payload's content *means* (whether a schema is sensible, a contract's terms are fair, a policy is wise). Any such evaluation belongs entirely to applications reading the payload.

6. Well-Formedness Rules

Each rule below is stated as a single testable condition: *a record is well-formed if and only if...* Rationale for these rules is not restated here — see Section 7 (Conformance Tests) for the constructions that motivated each one, and Section 8 for the theorems they support.

6.1 Authorization Traceability

A commitment or intention record is well-formed if and only if, at the time of its creation, a valid authorization path can be derived through the causal graph — either (a) a chain of causal references terminating in a commitment that grants the author the relevant capability, or (b) the record is a genesis record satisfying 6.5.

Absence of a valid authorization path SHALL be treated as absence of authority. A runtime MUST NOT treat an unresolvable authorization path as implicit grant.

Informative consequence (Mediated Influence). Applying this rule across authority boundaries yields a consequence not obvious from the rule's statement alone: one

authority's Intention cannot supply another authority's Intention with authorization merely by being cited as its cause. A citing Intention still requires its own resolvable authorization path under this rule; a causal reference to another authority's Intention is not, by itself, such a path. In practice this means cross-authority influence is only ever achieved through an intervening Observation (evidence the citing authority can act on) or an intervening Commitment (an explicit delegation), never through a bare Intention-to-Intention reference. This is not a separate rule — it is what 6.1 already requires, made explicit here because the failure mode it prevents (authority forgery via causal reference) was not obvious until adversarially constructed (see 7.6, Attack 1).

6.2 Capability Transitivity

A capability grant (including but not limited to a Delegation payload) is well-formed only if its transitivity is explicitly declared. In the absence of an explicit declaration, a capability grant SHALL be treated as non-transitive: the grantee MAY exercise the capability but MAY NOT grant it to a third party.

6.3 Observation Origin

Every `observation` record MUST declare exactly one origin value: `witnessed` (the author directly perceived the asserted state) or `inferred` (the asserted state was derived by rule from other records, such as an expiry condition). An observation record with no declared origin, or more than one, is malformed.

6.4 Schema Reference

Any payload whose correct interpretation depends on domain-specific vocabulary MUST reference exactly one Schema payload (5.2) defining the terms it uses. A payload requiring domain interpretation that carries no schema reference is malformed. Comparison or aggregation across two payloads referencing different, unmapped schemas is undefined (see 8.2.4, Local Meaning) — a runtime is not required to reject such a comparison, but MUST NOT silently coerce it to a shared interpretation.

Schema mapping. A relationship between two schemas (e.g., "Community A's `household` corresponds to Community B's `household` under these conditions") is expressed as an ordinary Commitment payload, causally referencing both schemas it relates. It receives no special runtime treatment beyond the envelope validation applied to any Commitment (5.1). The runtime does not recognize "mapping" as a distinct payload type — this is a direct application of the Minimal Elevation Principle (2.6): a mapping is expressible using existing primitives, so it is not elevated into the ontology or given its own well-formedness rule.

6.5 Genesis Records

A record with zero causal predecessors is well-formed only if it explicitly declares itself as a genesis record and satisfies the self-authorization mechanism defined for its primitive type (typically: a valid cryptographic signature proving possession of the identity's private key, with no further chain required).

7. Conformance Tests

Each test below constructs a higher-level concept from the primitives, axioms, and rules in Sections 3–6 alone, and states the result. A conforming implementation is not required to natively support any of the constructed concepts (Identity, Reputation, Contract, etc.) — it is required only to support the operations each construction is built from. These tests exist to demonstrate that support for those operations is sufficient.

7.1 Identity

Depends on: 4.1 (Immutability), 4.2 (Causality), 4.5 (Non-Silent Conflict), 6.1 (Authorization Traceability), 6.5 (Genesis Records)

Construction: A genesis Commitment self-authorizes a key-to-subject binding. Subsequent rotation/delegation Commitments causally reference and supersede it. Third-party belief in the binding accumulates as Observations, queried as a decayed confidence score, never as a Commitment.

Result: Derives without a new primitive. Requires two amendments to the invariants as originally stated: a genesis base case for Causality (now 4.0/6.5) and explicit provision for self-authorization alongside delegated authorization (6.1).

Produces: Theorem 8.2.5 (Bounded Continuity) — identity continuity across total key loss without a causal link is never provable by the runtime, only approached by accumulated Observation.

7.2 Reputation

Depends on: 4.1 (Immutability), 4.2 (Causality)

Construction: Observations record actions; Commitments (by whoever holds that authority) record evaluations of those actions referencing the Observation. "Current reputation" is not a record — it is a query summing/decaying the referencing Commitments at read time.

Result: Derives without a new primitive.

Produces: Confirms the general pattern of Theorem 8.2.2 (Transient Interpretation) — see 8.2.1–8.2.2 for the general statement this and every subsequent construction instantiate.

7.3 Contracts

Depends on: 6.1 (Authorization Traceability), 6.2 (Capability Transitivity), 8.2.2 (Transient Interpretation)

Construction: Party A's Commitment ("offer terms T") and Party B's causally-referencing Commitment ("accept terms T") together satisfy a query pattern. "Contract validity" is that query result, not a stored fact. Breach is an Observation referencing the acceptance; termination is a further Commitment, valid or not according to policy, not according to the runtime.

Result: Derives without a new primitive. Surfaces an implementation fork not present in earlier constructions: offer/acceptance as two independent Commitments (dangling-offer possible) versus a single co-signed Commitment (atomic, no dangling state) are both valid instantiations; this specification does not mandate one over the other.

Produces: Corollary to 8.2.4 (Local Meaning) — interpretation of a record can change even when the record does not, because interpretation is never itself a record and is therefore never subject to 4.1.

7.4 Workflow

Depends on: 4.2 (Causality), 4.4 (Closure), 4.5 (Non-Silent Conflict)

Construction: A root Intention causally spawns child Intentions. Retry is a new Intention referencing a terminal one, never a reopened one (3.4). A missed deadline with no witnessing party produces an inferred-origin Observation (6.3) authored by the runtime's own clock, satisfying Closure.

Result: Derives without a new primitive.

Produces: Theorem 8.1.3 (Acyclicity); Theorem 8.1.5 (Closure Is Strictly Local — the runtime guarantees only that each Intention emits its own terminal Observation, never that a parent's closure depends on its children's); confirms 4.5 applies to concurrent Intentions, not only concurrent Commitments.

7.5 Multi-Agent Coordination

Depends on: 4.3 (Local Sufficiency), 4.5 (Non-Silent Conflict), 6.1 (Authorization Traceability)

Construction: Independent authorities each hold independent Intention trees, connected only through shared Observations and authorizing Commitments — never through direct cross-authority Intention-to-Intention reference (see 6.1, Mediated Influence). Negotiation reduces to repeated instances of 7.3. Deadlock and over-allocation are recorded, not prevented, and become visible only as later Observations.

Result: Derives without a new primitive.

Produces: Theorem 8.2.6 (Emergent Coordination — no coordination record exists; coordinated behavior is a replay of the graph under a chosen policy); Theorem 8.2.3 (Policy Neutrality, restated formally from Design Principle 2.5); confirms the Mediated Influence consequence of 6.1.

7.6 Capability Security

Depends on: 6.1 (Authorization Traceability), 6.2 (Capability Transitivity)

Construction (adversarial): An attacker forms an Intention causally referencing a legitimate party's Intention or Observation, with no authorizing Commitment of their own.

Authorization traversal (6.1) finds no path and rejects the record. A second attack attempts unauthorized re-delegation; rejected by 6.2's non-transitive default.

Result: This test found the one genuine gap in the derivation program: Intentions were not originally required to satisfy 6.1 on the same footing as Commitments (causal reference to a legitimate record was being mistaken for authorization to act). Closed by symmetry correction, stated directly in 3.4 and 6.1 as written — not by adding a primitive.

Produces: Theorem 8.1.4 (Authorization Traversal Fails Safe — absence of a valid path is always treated as denial, never as implicit grant).

7.7 Semantics

Depends on: 6.4 (Schema Reference), 2.6 (Minimal Elevation)

Construction: A Schema Commitment defines a vocabulary. Payloads reference exactly one. Two communities with differing definitions produce no runtime error; comparison across them is undefined absent an explicit mapping Commitment (6.4), which is itself an ordinary payload, not a distinct type.

Result: Derives without a new primitive and without a dedicated semantics layer.

Produces: Theorem 8.2.4 (Local Meaning — no record is interpretable independent of a schema reference; the runtime provides no global semantic function).

8. Derived Theorems

Every theorem in this section is a consequence of Sections 3–6. None is independently checkable (1.4) — removing a theorem from this section changes nothing about what a conforming implementation must do. Theorems are included because they explain what conformance guarantees, for readers who are not tracing the kernel by hand.

Two kinds of theorem appear here, and the distinction matters for how each is used:

- **Structural theorems** (8.1) are statements about the causal graph itself — its shape, its integrity, its traversal properties. They hold regardless of what any application built on the runtime chooses to do.
- **Emergent theorems** (8.2) are statements about what applications built on the kernel necessarily inherit, whether or not they intend to. They concern behavior, not topology.

Answering "what does the runtime guarantee" points to 8.1. Answering "what does my application get for free" points to 8.2.

8.1 Structural Theorems

8.1.1 Explainability

Depends on: 4.1 (Immutability), 4.2 (Causality)

Claim: Every record's causal justification can be traced to completion — either to a Genesis Record or to a chain of causal predecessors, none of which have been altered or removed.

Proof sketch: By 4.2, every non-genesis record references at least one causal predecessor. By 4.1, no record referenced this way can have been modified or deleted since. Therefore the reference is always resolvable, and tracing terminates only at a Genesis Record (4.0), never at a broken or missing link.

8.1.2 Convergence

Depends on: 4.1 (Immutability), 4.3 (Local Sufficiency)

Claim: Synchronization between nodes may only add records to a node's view of the graph. It can never cause a node to lose a record it already validly held.

Proof sketch: By 4.1, no node can be instructed to delete or modify a record it holds — immutability is not a property one node can waive on another's behalf. By 4.3, a node's local state is never contingent on remote state. Therefore the only operation synchronization can perform is the union of two valid record sets, which is monotonically additive.

8.1.3 Acyclicity

Depends on: 4.1 (Immutability), 4.2 (Causality)

Claim: The causal graph is a directed acyclic graph.

Proof sketch: By 4.2, a record may only reference causal predecessors that already exist at the time of its creation. By 4.1, no record can later be altered to introduce a reference it did not originally have. Therefore no record can ever come to reference a record that causally follows it, which is precisely the condition for acyclicity. A corollary: every construct that appears iterative at the application level (retry, negotiation, loop) is graph expansion — a chain of new nodes — never a cycle in the stored structure (see 7.4).

8.1.4 Authorization Traversal Fails Safe

Depends on: 6.1 (Authorization Traceability)

Claim: If no valid authorization path exists for a Commitment or Intention, the record is denied — never implicitly granted.

Proof sketch: 6.1 defines well-formedness for these primitives as the existence of a resolvable authorization path. Absence of such a path is, definitionally, failure of the well-formedness condition, not a distinct or ambiguous outcome. There is no code path in a conforming implementation in which "no path found" resolves to acceptance; well-formedness is a boolean condition on a search result, not a default.

8.1.5 Closure Is Strictly Local

Depends on: 4.4 (Closure)

Claim: The runtime guarantees only that each individual Intention emits its own terminal Observation. It makes no guarantee about aggregation across an Intention hierarchy (e.g., that a parent's closure depends on, or is implied by, its children's).

Proof sketch: 4.4 is stated per-record: "every Intention which reaches a terminal state has at least one Observation... referencing it." Nothing in 4.4 references parent-child relationships, and no other invariant in Section 4 introduces hierarchy-aware closure semantics. Any such dependency is therefore necessarily an application-level policy querying multiple Observations, not a runtime guarantee (see 7.4).

8.2 Emergent Theorems

8.2.1 Query-State Separation

Depends on: 4.1 (Immutability), specifically the Notes in 4.1 ("interpretations are not records")

Claim: Persistent records and interpreted state are disjoint domains. Nothing computed from the graph at query time is itself a record, and nothing subject to 4.1's guarantees can be a query result.

Proof sketch: 4.1 constrains only records accepted into the store. A value produced by a function reading the store — a sum, a decay-weighted average, a pattern match — is not itself accepted into the store by that act of computation; it exists only for the duration of the query unless a separate act explicitly commits it as a new record. Therefore no computed value inherits 4.1's guarantees, and no computed value can be mistaken for a persistent fact without an explicit act of commitment.

Significance: This is the general law underlying every "X is a query, not a record" result in Section 7 (Reputation, Identity confidence, Contract validity, Resource availability). The three theorems immediately following are corollaries of this one, not independent results.

8.2.2 Transient Interpretation (corollary of 8.2.1)

Claim: Any higher-level concept computed from the graph (a score, a status, a validity determination) may change on every query, even when no new record has been added, if the computing function itself changes (e.g., a decay curve's time-dependence, a policy update).

Proof sketch: By 8.2.1, such a concept is not a record and carries no immutability guarantee. Its value at query time Q is a function of both the graph state and the computing function in effect at Q; either input changing changes the output, with no violation of any Section 4 invariant.

8.2.3 Policy Neutrality (corollary of 8.2.1; formalizes Design Principle 2.5)

Claim: The runtime never determines which of two conflicting records is correct, which goal has priority, or which interpretation of a payload is binding. Every such determination is a query result, computed by an application-supplied function.

Proof sketch: By 4.5, conflicting records are both preserved, unresolved, by the runtime itself — no invariant in Section 4 or rule in Section 6 selects a winner. By 8.2.1, any resolution that does occur is necessarily a query, external to the store. Therefore resolution is always attributable to an application's policy function, never to the runtime.

8.2.4 Local Meaning (*specialization of 8.2.1 for schema-dependent queries*)

Depends on: 6.4 (Schema Reference), 8.2.1 (Query-State Separation)

Claim: No record has meaning independent of a schema reference. The runtime provides no global semantic function; interpretation of any domain payload is itself a query, parameterized by whichever schema (and mapping, if applicable) the querying application chooses to apply.

Proof sketch: By 6.4, any payload requiring domain interpretation must reference a schema; comparison across differing, unmapped schemas is explicitly undefined. By 8.2.1, the act of interpreting a payload against a schema is a query, not a record. Therefore meaning is always relative to a schema chosen at query time, never fixed at write time by the runtime.

8.2.5 Bounded Continuity

Depends on: 4.2 (Causality), 6.1 (Authorization Traceability), 6.5 (Genesis Records)

Claim: Identity continuity across a complete break in the causal chain (e.g., total loss of all keys, with no delegation or rotation record connecting old to new) is never provable by the runtime — only approachable, asymptotically, through accumulated third-party Observation.

Proof sketch: By 6.1, a new Genesis Record has no causal predecessor by definition (4.0), and therefore no authorization path can connect it to a prior identity's history. Any claim that the two are "the same person" is necessarily an Observation (a belief), never a Commitment, since no party has the standing to constitute that continuity as fact. By 8.2.1, confidence in this claim is a query over accumulated Observations, asymptotic but never absolute (see 7.1).

8.2.6 Emergent Coordination

Depends on: 4.3 (Local Sufficiency), 4.5 (Non-Silent Conflict), 8.2.1 (Query-State Separation)

Claim: No "coordination" record exists anywhere in a conforming graph. Coordinated behavior among independent authorities is a replay of the graph — Observations, Commitments, and Intentions produced independently — interpreted under a chosen policy at query time. Different policies may derive different coordinated behaviors from the same immutable history.

Proof sketch: By 4.3, each authority's records are created independently, with no requirement to synchronize before acting. By 4.5, conflicting intentions among authorities are preserved, not merged into a single resolved state. By 8.2.1, any notion of "the plan" or

"the coordinated outcome" is therefore a query result, not a record, and is necessarily a function of whichever policy computes it (see 7.5).

9. Boundary Theorems

Each statement below is a **non-goal**: something this specification deliberately does not attempt, stated as a scope boundary rather than a shortcoming. A conforming implementation is not deficient for failing to provide any of these — providing them would mean it is solving a different problem than this specification defines. Where a theorem in Section 8 makes a boundary a mathematical certainty, this section states the boundary as policy; both readings are given because a reader may reasonably ask either "is this possible" or "is this intended."

9.1 Availability Boundary

Follows from: 4.3 (Local Sufficiency)

Claim: Global synchronous coordination as a precondition for local progress is outside the scope of this runtime.

Implication: Consensus protocols requiring contact with a majority of nodes before a write is valid (e.g., Raft, Paxos) do not conform to this specification and are not partial implementations of it — they solve a different problem (linearizable consistency under partition) than this runtime solves (availability under partition). A system requiring strict linearizability should be built as a component that consumes from this runtime via an adapter, not as an extension of it.

9.2 No Rollback

Follows from: 4.1 (Immutability)

Claim: Reversal of persistent history is outside the scope of this runtime.

Implication: Undoing an effect requires a new record — a compensating Commitment, a superseding Observation, a terminal Intention followed by a fresh one. This specification provides no operation that erases or rewinds prior state, by design, not by omission.

9.3 No Conflict Prevention

Follows from: 4.3 (Local Sufficiency), 4.5 (Non-Silent Conflict)

Claim: Preventing two authorities from creating mutually exclusive records is outside the scope of this runtime.

Implication: The runtime's guarantee is conflict *visibility*, not conflict *avoidance* — both records are preserved and surfaced, never silently merged or blocked at write time. A system requiring conflicts to be prevented before they occur (e.g., locking, reservation systems with guaranteed exclusivity) requires synchronous coordination and falls under the same boundary as 9.1.

9.4 No Global Semantics

Follows from: 6.4 (Schema Reference), 8.2.4 (Local Meaning)

Claim: Assigning universal, runtime-recognized meaning to any payload is outside the scope of this runtime.

Implication: This is a stronger claim than "different communities may use different schemas." It states that meaning itself is never fixed by the runtime at write time — every interpretation is relative to a schema (and, where relevant, a mapping) chosen by the querying application. No conforming implementation should attempt to resolve semantic disagreement on an application's behalf.

9.5 No Enforcement

Follows from: 6.1 (Authorization Traceability), 1.5 (Interpretation)

Claim: Compelling real-world actors to obey what is recorded as authorized or intended is outside the scope of this runtime.

Implication: The runtime records that an Intention was authorized at creation time (6.1) and records outcomes (4.4). It has no mechanism, and this specification defines none, for preventing an unauthorized real-world action from occurring, or for compelling an authorized one to occur. Authorization Traceability governs record acceptance, not physical or economic enforcement. This closes, permanently, the question raised during this specification's drafting process about whether Intentions require authorization "before execution" — they require it before *record creation*, which is the only event the runtime has jurisdiction over.

9.6 No Proven Identity Recovery

Follows from: 6.5 (Genesis Records), 8.2.5 (Bounded Continuity)

Claim: Proving that a new Genesis Record and a prior, causally-disconnected identity refer to the same underlying party is outside the scope of this runtime.

Implication: This is an impossibility result, not an implementation gap. No conforming implementation, however sophisticated, can supply a causal link that does not exist. Continuity across such a break can only ever be approached through accumulated third-party Observation (8.2.5) — never established as a Commitment, because no party has standing to constitute someone else's prior identity as fact.

10. Reference Runtime

Status: Deferred.

This specification intentionally separates the normative runtime (Sections 1–9) from any reference implementation. Writing a reference runtime before implementation experience exists risks smuggling unexamined design decisions into a document that has, until now, only contained claims that survived adversarial derivation.

A conforming reference runtime SHALL be developed after this specification reaches a stable v1.0. Implementation failures — cases where the specification is ambiguous, contradictory, or under-specified when actually built against — MUST be filed as counterexamples against the frozen text (see amendment process, header) and drive specification revisions. The specification is not to be silently adjusted to match whatever the first implementation happens to do.